

Avaliação Prática — Banco de Dados Avançado

Oct 5, 2026 · @amalgama

Sistema de Gestão de Excursões Turísticas

Tempo estimado: 6 horas | SGBD: Microsoft SQL Server | Valor: 10,0 pontos

1. Contexto

Uma agência de turismo precisa implementar, no banco de dados, as funcionalidades de pré-reserva de assentos, inscrição de clientes em excursões e controle das parcelas do pacote. A estrutura física (arquivo `01_estrutura_turismo_sqlserver.sql`, que também cria o banco **Turismo**) e uma massa inicial de dados (arquivo `02_dados_iniciais_turismo_sqlserver.sql`) são fornecidas pelo professor e devem ser executadas, nessa ordem, antes do início da avaliação. Para restaurar os dados originais a qualquer momento, execute novamente o arquivo 01 e depois o 02. Não altere tabelas ou colunas para adaptar o banco à sua solução.

Datas da avaliação: a massa inicial não usa datas fixas. Todas as datas (saídas, pré-reservas, inscrições, vencimentos e pagamentos) são calculadas a partir do dia em que o arquivo 02 é executado. Execute os testes no mesmo dia em que carregar a massa; em outro dia, recarregue 01 e 02. Nas regras que dependem do dia de hoje, considere a data atual como data de referência. Como as datas variam, os testes devem localizar os registros por consulta (por exemplo, "a primeira parcela ABERTA com vencimento futuro da inscrição 1"), e não por datas digitadas.

Fluxo principal: Cliente → Pré-reserva de assentos → Confirmação da inscrição → Geração de parcelas → Pagamento.

O cancelamento da inscrição depende da data de saída da excursão, da situação financeira do cliente e, quando houver parcelas vencidas, da situação de um acordo.

As situações de pré-reserva, inscrição, parcela e acordo ficam registradas em tabelas próprias de situação. Sempre que este enunciado citar uma situação em letras maiúsculas (por exemplo, AGUARDANDO, ATIVA ou PAGA), ela se refere ao registro correspondente nessas tabelas.

2. Regras de negócio

RN01 — Pré-reserva de assentos

- O cliente e a excursão informados devem existir e estar ativos. O pacote ao qual a excursão pertence também deve estar ativo.
- A quantidade de lugares solicitada deve estar entre 1 e 4.
- A excursão deve sair em, no mínimo, 15 dias contados da data da pré-reserva.
- A excursão deve possuir assentos disponíveis (RN02) em quantidade igual ou superior à quantidade de lugares solicitada.
- O cliente não pode ter, para a mesma excursão, outra pré-reserva AGUARDANDO ainda não expirada nem uma inscrição ATIVA.
- A pré-reserva criada com sucesso assume a situação AGUARDANDO, com data de expiração igual à data da pré-reserva + 5 dias.
- Pré-reserva gravada como AGUARDANDO, mas com a data de expiração anterior à data de referência, é considerada expirada: não impede nova pré-reserva do mesmo cliente na mesma excursão, não ocupa assentos e não pode ser confirmada.

RN02 — Controle de assentos

- Assentos disponíveis = quantidade de assentos da excursão menos os assentos ocupados.
- Ocupam assentos, **pela quantidade de lugares de cada registro** (e não pela contagem de registros): as inscrições ATIVAS da excursão e as pré-reservas AGUARDANDO ainda não expiradas.
- Não ocupam assentos como pré-reserva: as CONFIRMADAS (já representadas pela inscrição), CANCELADAS, EXPIRADAS e as AGUARDANDO vencidas.
- O resultado nunca pode ser inferior a zero.

RN03 — Confirmação da inscrição

- Somente pré-reserva AGUARDANDO e ainda não expirada pode ser confirmada.
- O chamador informa a quantidade de parcelas desejada, entre 1 e 10.
- Todas as parcelas devem vencer antes da data de saída da excursão. Se a última parcela, calculada conforme a RN04, vencer na data de saída ou depois dela, a confirmação é recusada.
- A confirmação cria uma inscrição ATIVA com o mesmo cliente, a mesma excursão e a mesma quantidade de lugares da pré-reserva. O valor total é o valor atual por pessoa do pacote multiplicado pela quantidade de lugares.
- A pré-reserva passa para a situação CONFIRMADA.

- São geradas as parcelas conforme a RN04.
- Essas alterações dependem umas das outras e devem ser atômicas: ou todas são gravadas, ou nenhuma é (ROLLBACK).

RN04 — Parcelas

- Cada inscrição possui exatamente a quantidade de parcelas registrada nela, numeradas de 1 até essa quantidade, sem repetição.
- O valor de cada parcela é o valor total dividido pela quantidade de parcelas, **truncado** em duas casas decimais. A diferença de centavos fica na última parcela, de modo que a soma das parcelas seja exatamente igual ao valor total.
- Exemplo: R\$ 1.850,00 em 3 parcelas gera R\$ 616,66, R\$ 616,66 e R\$ 616,68.
- A primeira parcela vence no dia 5 do mês seguinte ao da inscrição; cada parcela seguinte vence um mês após a anterior.
- Toda parcela inicia na situação ABERTA.
- O valor do pacote pode ser reajustado. A inscrição guarda o valor total da época da confirmação, e as parcelas usam sempre o valor gravado na inscrição.

RN05 — Pagamento

- Somente parcela ABERTA ou VENCIDA pode receber pagamento. Parcela PAGA ou CANCELADA não pode.
- A data do pagamento é informada pelo chamador (padrão: data atual) e não pode ser futura nem anterior à data da inscrição.
- O valor pago deve ser igual ou superior ao valor atualizado da parcela (RN06), calculado para a data do pagamento.
- O pagamento deve ser registrado com a parcela, a data e o valor pago.
- A mudança da parcela para PAGA não é feita pelo registro do pagamento, e sim pelo Trigger obrigatório (item 3.4).

RN06 — Encargos por atraso

- Sem atraso (pagamento até o dia do vencimento, inclusive), o valor atualizado é igual ao valor original.
- Com qualquer atraso, aplicam-se **multa fixa de 2%** sobre o valor original, cobrada uma única vez, e **juros simples de 1%** sobre o valor original para cada mês de atraso.
- Cada período mensal iniciado após o vencimento conta como um mês inteiro: 1 dia após o vencimento já conta 1 mês; 1 mês e 1 dia após, 2 meses; e assim por diante.
- O valor atualizado é arredondado para duas casas decimais.

- Exemplo: parcela de R\$ 500,00 com vencimento em 05/03. Paga em 05/03, vale R\$ 500,00; em 06/03 ou em 05/04, vale R\$ 515,00 (500 + 10 de multa + 5 de juros); em 06/04, vale R\$ 520,00.

RN07 — Parcela vencida

- A situação gravada na parcela pode estar desatualizada. Em consultas e cálculos, uma parcela ABERTA com vencimento anterior à data de referência deve ser tratada como vencida, mesmo que continue gravada como ABERTA.
- Data de referência é a data para a qual a consulta ou o cálculo é feito (por exemplo, a data do pagamento ou a data atual).

RN08 — Cancelamento da inscrição

- Somente inscrição ATIVA pode ser cancelada, e somente antes da data de saída da excursão (saída posterior à data atual).
- Parcelas pendentes são as que não foram pagas nem canceladas (ABERTAS ou VENCIDAS). Entre elas, são vencidas as que se enquadram na RN07 para a data atual.
- Sem parcelas pendentes vencidas, o cancelamento é permitido, mesmo que existam parcelas a vencer.
- Com parcelas pendentes vencidas, o cancelamento só é permitido se a inscrição tiver ao menos um acordo QUITADO, mesmo que existam outros acordos. Acordo ABERTO ou ROMPIDO, ou ausência de acordo, bloqueia o cancelamento.
- Cancelar significa: alterar a situação da inscrição para CANCELADA e alterar para CANCELADA todas as parcelas pendentes dessa inscrição. As duas alterações devem ser atômicas.
- O histórico deve ser preservado: não exclua inscrição, parcelas, pagamentos ou acordos. Parcelas PAGAS permanecem PAGAS.

3. Objetos obrigatórios

3.1 Views

- **VIEW 01 — Inscrições ativas:** listar somente as inscrições ATIVAS, identificando cliente, pacote, destino, código da excursão, data de saída, quantidade de lugares, valor total, quantidade de parcelas e situação.
- **VIEW 02 — Situação financeira:** apresentar uma linha por inscrição (de qualquer situação), com o cliente, o código da excursão, o total de parcelas, as quantidades de parcelas pagas, canceladas, pendentes e vencidas (RN07, na data atual), o valor

original ainda pendente e o valor atualizado ainda pendente (RN06, na data atual). Inscrições sem parcelas pendentes devem exibir zero, e não NULL, nos totais.

3.2 Functions

- **FUNCTION 01 — Assentos disponíveis:** receber uma excursão e uma data de referência e retornar quantos assentos ela ainda possui, conforme a RN02. Excursão inexistente retorna NULL.
- **FUNCTION 02 — Valor atualizado da parcela:** receber uma parcela e uma data de referência e retornar quanto a parcela vale nessa data, com multa e juros quando houver atraso (RN06 e RN07). Parcela inexistente retorna NULL; parcela PAGA ou CANCELADA retorna 0,00 (nada a pagar).

3.3 Stored Procedures

- **PROCEDURE 01 — Registrar pré-reserva:** receber cliente, excursão e quantidade de lugares; validar a RN01 e a RN02, registrar a pré-reserva e devolver o Id criado em um parâmetro OUTPUT.
- **PROCEDURE 02 — Confirmar inscrição:** receber a pré-reserva e a quantidade de parcelas; validar a pré-reserva e o prazo das parcelas, criar a inscrição, alterar a situação da pré-reserva e gerar as parcelas (RN03 e RN04), com controle transacional. Devolver o Id da inscrição em um parâmetro OUTPUT.
- **PROCEDURE 03 — Registrar pagamento:** receber a parcela, o valor pago e a data do pagamento (opcional); validar a RN05, calcular o valor atualizado com a FUNCTION 02 e registrar o pagamento. A situação da parcela não é alterada aqui: isso é responsabilidade do Trigger.
- **PROCEDURE 04 — Cancelar inscrição:** aplicar a RN08 e, quando permitido, cancelar a inscrição e suas parcelas pendentes em uma única transação.

Todas as procedures devem informar ao chamador o resultado da operação por um código de retorno: **0 indica sucesso** e cada motivo de recusa possui um código próprio, diferente de zero, descrito na documentação do objeto.

3.4 Trigger

- **TRIGGER 01 — Atualização da parcela após pagamento:** após INSERT na tabela de pagamentos, alterar para PAGA as parcelas correspondentes aos pagamentos registrados.
- Deve utilizar a tabela lógica `inserted`.
- Deve funcionar quando um único INSERT registrar pagamentos de mais de uma parcela: nunca assuma que `inserted` contém apenas uma linha.

- Deve funcionar como última barreira para INSERT feito diretamente na tabela (sem a procedure): se alguma parcela do lote já estiver PAGA ou CANCELADA, ou se o mesmo INSERT trouxer mais de um pagamento para a mesma parcela, a instrução inteira deve ser recusada com mensagem de erro e nenhum pagamento do lote pode permanecer gravado.
- Se a atualização da parcela falhar, o pagamento que a originou também não pode permanecer gravado.

4. Documentação obrigatória

Toda View, Function, Stored Procedure e Trigger deve possuir um bloco de documentação (comentário) junto ao objeto, para que qualquer pessoa entenda o objeto sem ler todo o código.

- Campos obrigatórios em todos os objetos: **Arquivo Fonte** (nome do arquivo em que o objeto está), **Objetivo**, **Autor**, **Data** e **Exemplo** (como executar ou consultar o objeto).
- Functions e Procedures devem informar também os **Parâmetros** e os **Retornos**: o que cada código ou valor retornado significa.
- Triggers devem descrever as situações de erro ou bloqueio.
- O exemplo dentro da documentação é apenas uma referência de uso e não substitui os testes ativos do item 6.

5. Qualidade e transações

- Declare somente variáveis que tenham finalidade e sejam de fato utilizadas no código.
- Identifique as situações pela coluna Descricao das tabelas de situação, e não por Ids digitados no código.
- As regras de assentos (RN02) e de valor atualizado (RN06) devem ser calculadas pelas Functions, e não reescritas dentro das procedures e views.
- Procedures com mais de uma escrita dependente (confirmação e cancelamento) devem confirmar (COMMIT) somente quando todas as etapas derem certo e desfazer tudo (ROLLBACK) em caso de erro.
- Não é exigida uma sintaxe específica de tratamento de erro, desde que o comportamento seja equivalente.
- As constraints do banco (chaves, UNIQUE e CHECK) são a última barreira, não a validação da regra. As procedures devem identificar os casos inválidos antes de gravar e devolver o código de retorno correspondente, em vez de deixar o erro do SQL Server interromper a execução.
- Não altere a estrutura física fornecida.

- Soluções diferentes são aceitas quando preservarem as regras de negócio.

6. Testes ativos obrigatórios

Os testes comprovam que a solução funciona. Eles devem estar ativos, ou seja, escritos como código que é executado: código de teste apenas comentado não será considerado. Monte os cenários a partir da massa inicial fornecida e, em cada teste, exiba o código de retorno obtido e o esperado.

- Pré-reserva válida e tentativas de pré-reserva: em excursão sem assentos suficientes, duplicada (mesmo cliente e mesma excursão, tanto por pré-reserva AGUARDANDO quanto por inscrição ATIVA), com cliente inativo, com excursão inativa, com pacote inativo, com saída em menos de 15 dias e com quantidade de lugares inválida.
- Pré-reserva válida para cliente que só possui, na mesma excursão, uma pré-reserva AGUARDANDO já expirada.
- Confirmação válida, conferindo quantidade, números, valores (inclusive a diferença de centavos na última parcela) e vencimentos das parcelas geradas, e que a soma das parcelas é igual ao valor total.
- Tentativas de confirmar uma pré-reserva já CONFIRMADA, uma pré-reserva com a data de expiração vencida e uma confirmação cuja última parcela venceria após a saída.
- Pagamento dentro do vencimento e pagamento em atraso (com multa e juros), além de uma tentativa com valor inferior ao atualizado.
- Tentativa de pagamento de parcela já PAGA.
- Teste do Trigger, comprovando que a parcela passou para PAGA após o pagamento.
- Teste do Trigger com várias linhas: uma única instrução INSERT registrando pagamentos de mais de uma parcela, comprovando que todas foram atualizadas.
- Teste do Trigger como barreira: um INSERT direto em parcela já PAGA, comprovando que o erro ocorreu e que nenhum pagamento do lote ficou gravado.
- Tentativas de cancelamento bloqueadas: com parcelas vencidas e sem acordo QUITADO (incluindo inscrição que só possui acordo ABERTO ou ROMPIDO) e de inscrição cuja excursão já saiu.
- Cancelamentos permitidos: com acordo QUITADO, sem parcelas pendentes e com parcelas pendentes apenas a vencer, comprovando que as parcelas pendentes passaram para CANCELADA e que as PAGAS foram preservadas.
- Chamadas das duas Functions, incluindo a FUNCTION 02 nas três datas-limite do exemplo da RN06 (dia do vencimento, um dia após e um mês e um dia após), e consultas às duas Views.

Os testes podem ser organizados com BEGIN TRANSACTION/ROLLBACK, para não alterar permanentemente a massa inicial e permitir que sejam executados novamente.

7. Entrega

A entrega não deve ser feita em um único arquivo. Cada objeto deve estar em um arquivo .sql próprio, guardado na pasta do seu tipo, como em um projeto real.

```
avaliacao_sql_nome_aluno/  
├─ Functions/    → FUNCTION 01 e FUNCTION 02  
├─ Views/        → VIEW 01 e VIEW 02  
├─ Procedures/   → PROCEDURE 01, 02, 03 e 04  
├─ Triggers/     → TRIGGER 01  
└─ Testes/      → testes ativos do item 6
```

- Cada arquivo deve ter o nome do objeto que contém, e esse nome deve ser o mesmo informado no campo Arquivo Fonte da documentação.
- Todos os arquivos devem executar sem erros em um banco criado com os arquivos oficiais 01 e 02, respeitando a ordem de dependência: Functions, Views, Procedures, Triggers e, por último, Testes.
- Tempo estimado para a execução da avaliação: 6 horas.
- Soluções semanticamente equivalentes são aceitas quando preservarem as regras.